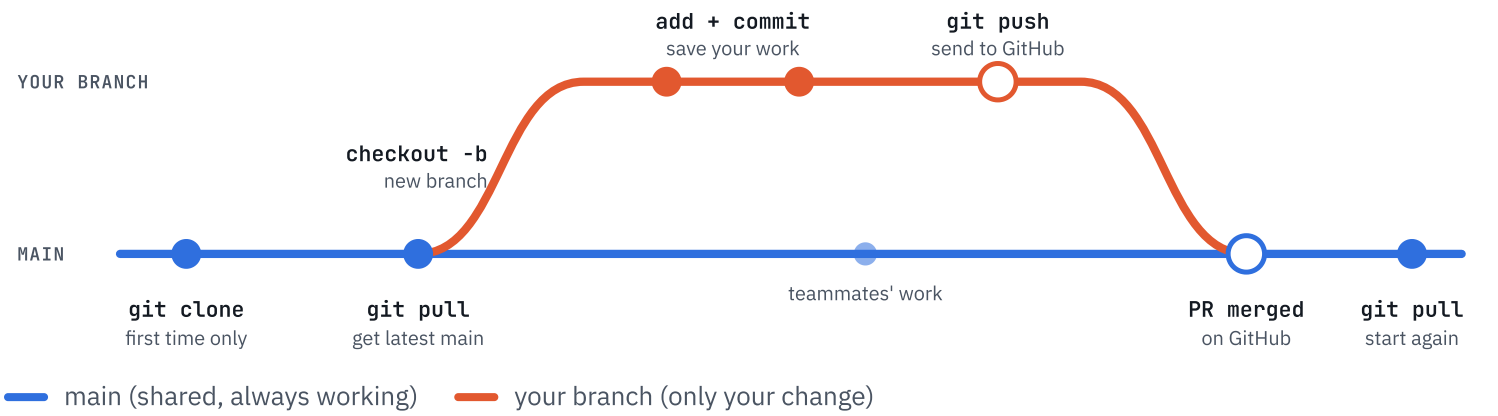
Git daily commands

Four commands cover the everyday work: get the project, get the latest code, start your own branch, and send your work back. The picture shows how they fit together.

Repo **github.com/prakash116/Marketing-Platform-Frontend** · default branch **main**



The four commands

1 Clone the project

Download the repo to your computer. Do this once.

```
terminal

$ git clone https://github.com/prakash116/Marketing-Platform-Frontend.git
$ cd Marketing-Platform-Frontend
$ npm install
$ cp .env.example .env.local   # then fill in the values
```

After this, `npm run dev` starts the app.

2 Pull the latest code

Bring your copy of main up to date with GitHub.

```
terminal

$ git checkout main
$ git pull origin main
$ npm install   # only if package.json changed
```

Do this every morning and before you start a new branch.

3 Create a branch

A separate line for your change, so main stays safe.

```
terminal

$ git checkout -b feat/header-menus
# other names: fix/login-error, docs/read
me
$ git branch   # check which branch you a
re on
```

Start the name with `feat/`, `fix/` or `docs/`.

4 Push your code

Save your changes and send the branch to GitHub.

```
terminal

$ git status   # see what changed
$ git add .
$ git commit -m "feat(header): add first
design of website and app headers"
$ git push -u origin feat/header-menus
```

Then open GitHub and click **Compare & pull request**. Next pushes on this branch are just `git push`.

Daily workflow, step by step

1. **Go to main**
   `git checkout main`
2. **Get the latest code**
   `git pull origin main`
3. **Create your branch**
   `git checkout -b feat/my-change`
4. **Write code and test it**
   `npm run dev`
   Before pushing: `npm run lint` and `npm test`
5. **Stage your changes**
   `git add .`
6. **Commit with a clear message**
   `git commit -m "feat: what you did"`
   Lint and Prettier run automatically on commit. If they fail, fix the errors and commit again.
7. **Push the branch**
   `git push -u origin feat/my-change`
8. **Open a pull request on GitHub**
   CI runs lint, typecheck, tests and build. After review, merge it into main.

Rules for this repo

Commit message format

Commitlint rejects any other format. Write `type(scope): message`. The scope in brackets is optional.

✓ `feat(header): add first design of website and app headers`
✓ `fix: close menu on Esc`
✓ `docs: update readme`
✗ `header: first design` “header” is not a type

`feat`	New feature
`fix`	Bug fix
`style`	Formatting only, no logic change
`refactor`	Cleaner code, same behaviour
`docs`	Docs or README
`test`	Tests
`chore`	Config, packages, small upkeep
`build` · `ci` · `perf` · `revert`	Build setup, CI, speed, undo a commit

Start the message in lowercase and keep the whole line under 100 characters.

9

Back to step 1 for the next task

```
git checkout main && git pull
```

Never push straight to main

Always work on a branch and merge through a pull request, so CI checks your code first.

Pull before you branch

Starting from an old main causes merge conflicts later.

Small commits

One idea per commit makes review easy and mistakes easy to undo.

Handy extras

`git status`

What changed, and which branch you are on.

`git log --oneline`

Short list of recent commits.

`git checkout main`

Switch to another branch.

`git stash` / `git stash pop`

Put unfinished work aside, then bring it back.

`git restore file.tsx`

Throw away changes to one file you have not committed.

`git pull origin main`

Run it on your branch to bring in teammates' new work.